**Robi Robot Final Project Write-Up**

**Table of Contents**



---

## 1. Introduction

The Robi robot project is designed to provide autonomous navigation capabilities to a Raspberry Pi-controlled robot. This project combines low-level hardware control using I2C communication and GPIO with high-level behavior logic for movement and safety. The robot's primary goals include forward movement, avoiding obstacles (walls), and detecting and avoiding ledges to prevent falls. This write-up documents the implementation of the custom C-based drivers and

APIs that manage Robi's behavior, along with the problems encountered and the rationale behind using multithreading.

---

**2. Driver/API Manual**

This section serves as a manual for developers who wish to use or build upon the motor and sensor drivers created for Robi.

**2.1 I2C Driver (I2C.c / I2C.h)**

- **i2cInit(address)**: Initializes the I2C interface and sets the slave address of the device.
- **write8(reg, value)**: Writes an 8-bit value to a specific register.
- **readU8(reg)**: Reads an 8-bit unsigned value from a specific register.

**2.2 PWM Driver (PWM.h)**

- **PWMInit(address)**: Initializes the PCA9685 PWM controller at the given I2C address.
- **setPWMFreq(freq)**: Sets the PWM frequency.
- **setPWM(channel, on, off)**: Sets the duty cycle for a specific channel.

- **setAllPWM(on, off)**: Sets duty cycles for all channels simultaneously.

### 2.3 MotorHat Driver (MotorHat.h)

- **initMotors()**: Initializes motor pin configurations.

- **initHat()**: Initializes the Motor HAT.

- **run(command, motorID)**: Runs the selected motor in the desired direction.

- **setPin(pin, value)**: Sets a pin HIGH or LOW using PWM.

### 2.4 Robot Control API (Robot.h)

- **robot_init(left_trim, right_trim)**: Initializes the robot with motor trim values.

- **robot_forward(speed)**, **robot_backward(speed)**: Drives the robot forward or backward.

- **robot_left(speed)**, **robot_right(speed)**: Spins the robot in place.

- **robot_stop()**: Stops all motor activity.

### 2.5 Sensor API (Sensor.h)

- **sensor_init()**: Initializes the GPIO pins connected to sensors.

- **wall_detected()**: Returns 1 if a wall is detected.

- **ledge_detected()**: Returns 1 if a ledge (drop) is detected.

---

**3. Developer Log: Problems and Solutions**

| Problem | Solution |
|---|---|
| Typo in I2C buffer variable | Changed buff to buf in write8() function |
| Incorrect use of bcm_i2c_write() | Replaced with bcm2835_i2c_write() |
| Missing read-before-write logic in readU8() | Added correct register select step before reading |
| Lack of threading to monitor sensors | Implemented multithreading using pthread to run sensor logic concurrently |
| No automatic redirection logic | Added movement logic in sensor thread to stop, reverse, turn, and resume |

| | |
|---|---|
| Undefined PWM behavior | Developed full inline implementations inside PWM.h and MotorHat.h to avoid new file creation |
| Sensor GPIO not responsive | Used pull-up resistors and confirmed pin mapping with Raspberry Pi documentation |

---

**4. Design Decisions: Why Multithreading?**

Multithreading was chosen over multiprocessing due to the need for efficient real-time response and shared memory access between control logic and sensor monitoring. Threads are lighter weight and allow the main movement thread and the sensor monitoring thread to communicate via shared flags without requiring complex inter-process communication.

For example, the sensor_mon thread runs continuously and updates a shared stop_robot flag if a wall or ledge is detected. The main thread reads this flag to determine whether to continue forward or to stop and redirect. Using pthread provided low-overhead concurrency and simple synchronization, which is ideal for embedded robotics on resource-constrained hardware like the Raspberry Pi.

---

**5. Conclusion**

The Robi robot project successfully integrates custom C drivers and APIs to support safe, autonomous movement. With a robust modular structure, this implementation provides an extensible foundation for future features such as remote control, camera integration, or mapping.

The use of multithreading allowed Robi to simultaneously move and monitor its surroundings, ensuring safe navigation without the need for complex multiprocessing. The lessons learned from solving various implementation issues have significantly improved the robustness and reliability of the system.

---

**6. Works Cited**


- Adafruit. "Adafruit Motor HAT for Raspberry Pi - Python Library." https://github.com/adafruit/Adafruit-Motor-HAT-Python-Library

- Broadcom. "BCM2835 C Library." https://www.airspayce.com/mikem/bcm2835/

- Raspberry Pi Documentation. "GPIO Usage." https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

- DiCola, Tony. "Simple Robot Control with Python." https://learn.adafruit.com/robotic-raspberry-pi-rover

- pthreads Tutorial. https://computing.llnl.gov/tutorials/pthreads/